



Software Requirement Engineering Notes (Based on Tips)

Section 1: General Software Engineering Concepts

Definition

Software Engineering (SE) is the application of engineering principles to the development, operation, and maintenance of software systems. It ensures:

- High quality
- Efficiency
- Systematic and disciplined approaches

Core Components

1. Processes:

Structured activities like Planning → Analysis → Design → Implementation → Testing → Maintenance.

2. Tools:

Support development (e.g., IDEs, version control, testing tools).

3. Methodologies:

Frameworks to structure processes (e.g., Agile, Waterfall, DevOps).

Key Activities in Requirements Engineering

1. **Elicitation** – Gathering requirements from stakeholders (interviews, surveys, etc.)
2. **Documentation** – Creating clear, verifiable Software Requirements Specifications (SRS)
3. **Validation** – Ensuring requirements are complete, feasible, and well-understood
4. **Management** – Updating requirements as the project evolves

Key Concepts

- **Goals:** Consistency, Quality, Communication, Complexity reduction, Risk mitigation
 - **Challenges:** Ambiguity, Incompleteness, Incorrect requirements → Costly delays
 - **Risks:** Feasibility issues, Scope creep
-

Section 2: Requirements Engineering

Requirements Inception

- Understand business needs and goals
- Identify key stakeholders
- Define high-level system scope
- Outcome: Clear understanding of the problem domain

Domain Understanding

- Define the **Product Vision**

Requirement Elicitation Techniques

- **Interviews:**
 - Structured
 - Semi-structured

- Unstructured
 - **Questionnaires:**
 - Open-ended
 - Closed-ended
 - **Observation:**
 - Direct
 - Participatory
 - **Document Reading Technique**
 - Manuals, technical specifications, historical records, policy documents.
-

Section 3: Specification and Modeling

SRS (Software Requirements Specification)

- A formal document that describes:
 - Functional & non-functional requirements
 - Constraints
 - Interface and system behavior

Modeling Techniques

- **Use Case Diagram**
 - Graphical representation of user interactions with the system
 - Key components: Actors, Use Cases, System boundaries
- **DFD Level 0 (Data Flow Diagram)**
 - Shows system's major processes, data flows, and data stores at a high level

Description vs. Specification

- **Description:** Informal, natural language

- E.g., "The system should allow users to login."
- **Specification:** Formal, structured, unambiguous
 - E.g., "The system shall authenticate users using a valid username and password stored in the database."

Formal Specification

- **Definition:** Use of mathematical models to define software behavior
 - **Benefits:**
 - Eliminates ambiguity
 - Improves verification & validation
 - **Methods:**
 - Z-Notation
 - B-Method
 - VDM (Vienna Development Method)
-

Section 4: Validation, Prioritization, and Safety Analysis

V&V (Verification and Validation)

- **Verification:** Are we building the product **right**?
 - Logical correctness of requirement documents
- **Validation:** Are we building the **right** product?
 - Do the requirements meet stakeholder needs?

Benefits:

- Reduces costly rework
 - Enhances quality
 - Ensures customer satisfaction
-

Requirement Prioritization: MoSCoW Method

- **Must Have** – Essential features
 - **Should Have** – Important but not critical
 - **Could Have** – Nice to have
 - **Won't Have** – Out of current scope
-

Safety Analysis Techniques

Hazard analysis is a systematic approach to identifying potential conditions or events that might lead to system failure or harm.

It helps address critical safety questions:

- What could potentially go wrong?
- What are the possible consequences?
- What preventive measures can be taken?

Conducted early in the development process, hazard analysis plays a key role in shaping clear and effective safety requirements.

1. FMEA (Failure Modes and Effects Analysis)

- **Approach:** Bottom-up
- **Goal:** Identify potential component failures, their effects, and prioritize by risk
- **Steps:**
 1. List components and functions
 2. Identify failure modes
 3. Analyze effects
 4. Assign Risk Priority Number (RPN)
 5. Recommend actions

Example:

- Component: ECG sensor

- Failure Mode: Sensor disconnect
 - Effect: No reading → patient risk
 - RPN: High → Add alert system
-

2. FTA (Fault Tree Analysis)

- **Approach:** Top-down
- **Goal:** Analyze causes of a major system failure using logic gates
- **Steps:**
 1. Define top undesired event
 2. Break down causes (AND, OR logic)
 3. Identify minimal cut sets
 4. Analyze probabilities

Example:

- Top Event: Aircraft loses power
 - Causes: Fuel exhaustion, software crash
-

3. HAZOP (Hazard and Operability Study)

- **Approach:** Structured team review using guide words
- **Goal:** Detect deviations from intended design and analyze their impact
- **Steps:**
 1. Form a team
 2. Divide system into nodes
 3. Apply guide words (e.g., "No", "More", "Less")
 4. Analyze causes and consequences
 5. Document necessary actions

Example:

- Node: Gas pipeline
- Deviation: "More pressure"
- Consequence: Explosion risk
- Action: Add pressure relief valve

Here's a structured **comparison** of **FMEA**, **FTA**, and **HAZOP** across key dimensions:

Comparison Table

Aspect	FMEA (Failure Modes and Effects Analysis)	FTA (Fault Tree Analysis)	HAZOP (Hazard and Operability Study)
Approach	Bottom-up	Top-down	Structured brainstorming (node-based)
Focus	Component-level failures and effects	System-level failure causes	Design deviations and process hazards
Advantage	- Systematic prioritization using RPN- Easy to update during design phase	- Visual logic representation- Helps identify root causes	- Encourages cross-disciplinary insight- Captures subtle process issues
Disadvantage	- Can miss system-level interactions- May overlook rare events	- Can be complex and time-consuming- Requires clear top event	- Requires experienced team- Less quantitative
Best for (Scenario)	Early design phase; component reliability (e.g., medical devices)	Post-incident analysis or critical system failure prevention (e.g., aviation safety)	Hazard analysis in chemical/process industries (e.g., refineries, pipelines)

Summary Recommendations

- **Use FMEA** when analyzing **component-level risks**, especially during **early development**.

- **Use FTA** when you have a **specific system failure** you want to prevent or diagnose.
- **Use HAZOP** when analyzing **process-based systems** for **unexpected deviations** or during **design reviews**.